

DIGITAL LIBRARY SYSTEM

Full CRUD Operations Assignment

1. Overview

In this assignment you will build a complete NestJS REST API that connects to an Oracle Enterprise database running on your local machine. You will implement full CRUD (Create, Read, Update, Delete) operations for two resources: Books and Users.

What you will build:

- A Books module with 5 CRUD endpoints backed by an Oracle table
-
- A Users module with 5 CRUD endpoints backed by an Oracle table
- A Pluggable Database (PDB) inside your Oracle Enterprise installation
- TypeORM entities that map your TypeScript classes to database tables
- DTOs (Data Transfer Objects) to validate incoming request data
- Repositories to handle all database operations
- An environment file to store sensitive connection settings

By the end of this assignment you will be able to send HTTP requests from Postman and verify that data is being created, read, updated, and deleted directly inside your Oracle database.

2. Prerequisites

Make sure all of the following are installed and working on your machine before you begin:

Requirement	How to verify
Oracle Database Enterprise Edition	Open SQL*Plus and log in as SYSTEM
Nestjs Project with the last assignment done	Open it in VS Code and run
Postman	Open the application

3. Step 1 — Create Your Pluggable Database (PDB)

What is a Pluggable Database?

WHY: Oracle Enterprise uses a Container Database (CDB) that holds many smaller Pluggable Databases (PDBs) inside it. Think of the CDB as a building, and your PDB as your own private office inside that building. You will create your own PDB so your application has a dedicated place to store data.

Open SQL*Plus on your machine and connect as SYSDBA:

```
sqlplus / as sysdba
```

Once connected, run these SQL commands one at a time:

3a. Check existing PDBs

```
SELECT name, open_mode FROM v$pdb;
```

This shows you all PDBs currently in your Oracle instance. You will add a new one next.

3b. Create your Pluggable Database

```
CREATE PLUGGABLE DATABASE library_pdb  
ADMIN USER pdb_admin IDENTIFIED BY admin_password  
FILE_NAME_CONVERT = (  
  'C:\oracle\oradata\ORCL\pdbseed\  
  'C:\oracle\oradata\ORCL\library_pdb\  
);
```

NOTE: Adjust the file path above to match where Oracle is installed on your PC. A common path is C:\oracle\oradata\ORCL. Run `SELECT name FROM v$datafile;` if you are unsure.

3c. Open the PDB

```
ALTER PLUGGABLE DATABASE library_pdb OPEN;
```

3d. Switch into your PDB

```
ALTER SESSION SET CONTAINER = library_pdb;
```

3e. Create an application user

```
CREATE USER library_user IDENTIFIED BY library_password;  
GRANT CONNECT, RESOURCE, DBA TO library_user;
```

3f. Verify the user was created

```
SELECT username FROM dba_users WHERE username = 'LIBRARY_USER';
```

You should see LIBRARY_USER in the results. Your database and user are now ready.

Read more about Pluggable Databases:

<https://docs.oracle.com/en/database/oracle/oracle-database/21/multi/introduction-to-the-multitenant-architecture.html>

4. Step 2 — Set Up Your NestJS Project

What are we doing?

WHY: We are installing the packages needed to talk to an Oracle database. TypeORM is the library that translates our TypeScript code into SQL. oracledb is the driver that actually connects to Oracle.

4a. Install required packages

- Open Git Bash from inside your **project folder** and run the following commands; otherwise, you will get an **error** if you run them from a different folder:

```
npm install @nestjs/typeorm typeorm oracledb  
npm install @nestjs/config  
npm install class-validator class-transformer
```

What each package does:

Package	Purpose
@nestjs/typeorm	Connects NestJS with TypeORM
typeorm	Converts TypeScript classes to database tables and SQL Links: https://typeorm.io/docs/getting-started
oracledb	The Oracle database driver — lets Node.js talk to Oracle
@nestjs/config	Reads your .env file and makes values available in code
class-validator	Validates incoming request data (e.g. checks required fields) Link: https://docs.nestjs.com/pipes#class-validator

Package	Purpose
class-transformer	Transforms plain JSON into TypeScript class instances Link: https://docs.nestjs.com/pipes#class-validator (it's also found under the same link)

bc. Generate modules, services, and controllers

- We assume everyone already did these commands

```
nest g mo books
```

```
nest g s books
```

```
nest g co books
```

```
nest g mo users
```

```
nest g s users
```

```
nest g co users
```

Read more about NestJS CLI commands: <https://docs.nestjs.com/cli/usage>

5. Step 3 — Create the Environment File (.env)

What is an .env file?

WHY: An .env file stores sensitive configuration values like passwords and hostnames outside of your source code. You never push this file to GitHub. Your application reads these values at runtime so you can change them without touching code.

Create a file called **.env** in the root of your project (same level as package.json) and add the following:

```
DB_HOST=localhost
DB_PORT=1521
DB_USERNAME=library_user
DB_PASSWORD=library_password
DB_SERVICE_NAME=library_pdb
DB_SYNCHRONIZE=true
PORT=3000
```

NOTE: DB_SYNCHRONIZE=true means TypeORM will automatically create or update your database tables whenever you start the application. This is very useful during development but should be set to false in production.

Also create a **.gitignore** file (or add to it) and make sure .env is listed:

```
.env
node_modules
```

Read more about `@nestjs/config` and environment variables:
<https://docs.nestjs.com/techniques/configuration>

6. Step 4 — Configure the Database in `app.module.ts`

What are we doing?

WHY: We are telling NestJS how to connect to our Oracle database. `TypeOrmModule.forRootAsync()` reads our `.env` values and opens a connection pool to Oracle when the app starts. `ConfigModule` makes `.env` values available everywhere.

Replace the contents of `src/app.module.ts` with the following:

```
import { Module } from '@nestjs/common';
import { ConfigModule, ConfigService } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { BooksModule } from './books/books.module';
import { UsersModule } from './users/users.module';
import { Book } from './books/entities/book.entity';
import { User } from './users/entities/user.entity';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRootAsync({
      imports: [ConfigModule],
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        type: 'oracle',
        host: config.get('DB_HOST'),
        port: parseInt(config.get('DB_PORT')),
        username: config.get('DB_USERNAME'),
        password: config.get('DB_PASSWORD'),
        serviceName: config.get('DB_SERVICE_NAME'),
        synchronize: config.get('DB_SYNCHRONIZE') === 'true',
        entities: [Book, User],
        logging: true,
      }),
    ),
    BooksModule,
```

```
    UsersModule,  
  ],  
})  
export class AppModule {}
```

Read more about TypeORM database connection:
<https://docs.nestjs.com/techniques/database>

7. Step 5 — Create Entities

What is an Entity?

WHY: An Entity is a TypeScript class decorated with `@Entity()`. TypeORM reads this class and creates a matching table in your Oracle database. Each property decorated with `@Column()` becomes a column in that table. Think of the class as a blueprint for your table.

7a. Book Entity

Create the file: `src/books/entities/book.entity.ts`

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn } from 'typeorm';  
  
@Entity('books')  
export class Book {  
  
    @PrimaryGeneratedColumn()  
    id: number;  
  
    @Column({ length: 255 })  
    title: string;  
  
    @Column({ length: 255 })  
    author: string;  
  
    @Column()  
    year: number;  
  
    @Column({ length: 100, nullable: true })  
    genre: string;
```

```
@CreateDateColumn()
createdAt: Date;
}
```

Decorator meanings:

Decorator	What it does
@Entity('books')	Maps this class to a table called BOOKS in Oracle
@PrimaryGeneratedColumn()	Auto-incrementing primary key column (ID)
@Column()	A regular column in the table
@CreateDateColumn()	Automatically records the date/time a row was inserted

7b. User Entity

Create the file: src/users/entities/user.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn } from 'typeorm';
```

```
@Entity('users')
export class User {
```

```
  @PrimaryGeneratedColumn()
  id: number;
```

```
  @Column({ length: 255 })
  name: string;
```

```
  @Column({ unique: true, length: 255 })
  email: string;
```

```
  @Column({ length: 50, default: 'member' })
  role: string;
```

```
  @CreateDateColumn()
  createdAt: Date;
}
```

NOTE: The *unique: true* option on email tells Oracle to create a **UNIQUE** constraint on that column — no two users can have the same email address.

Read more about TypeORM entities and decorators: <https://typeorm.io/entities>

8. Step 6 — Create DTOs (Data Transfer Objects)

What is a DTO?

WHY: A DTO is a TypeScript class that defines exactly what data is allowed into your API. When someone sends a POST or PATCH request, the DTO checks that all required fields are present and have the correct types. This protects your database from receiving bad or incomplete data.

8a. Book DTOs

Create the file: `src/books/dto/create-book.dto.ts`

```
import { IsString, IsNumber, IsOptional, IsNotEmpty } from 'class-validator';

export class CreateBookDto {
  @IsString()
  @IsNotEmpty()
  title: string;

  @IsString()
  @IsNotEmpty()
  author: string;

  @IsNumber()
  year: number;

  @IsString()
  @IsOptional()
  genre?: string;
}
```

Create the file: `src/books/dto/update-book.dto.ts`

```
import { PartialType } from '@nestjs/mapped-types';
import { CreateBookDto } from './create-book.dto';
```



```
export class UpdateBookDto extends PartialType(CreateBookDto) {}
```

NOTE: *PartialType makes ALL fields from CreateBookDto optional. This is perfect for PATCH requests where you only want to update some fields, not all of them.*

8b. User DTOs

Create the file: `src/users/dto/create-user.dto.ts`

```
import { IsString, IsEmail, IsOptional, IsNotEmpty } from 'class-validator';

export class CreateUserDto {
  @IsString()
  @IsNotEmpty()
  name: string;

  @IsEmail()
  email: string;

  @IsString()
  @IsOptional()
  role?: string;
}
```

Create the file: `src/users/dto/update-user.dto.ts`

```
import { PartialType } from '@nestjs/mapped-types';
import { CreateUserDto } from './create-user.dto';

export class UpdateUserDto extends PartialType(CreateUserDto) {}
```

Enable validation globally by adding this to `src/main.ts`:

```
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(new ValidationPipe({ whitelist: true }));
  await app.listen(process.env.PORT || 3000);
}
```

```
}  
bootstrap();
```

Read more about DTOs and validation: <https://docs.nestjs.com/techniques/validation>

Read more about class-validator decorators:
<https://github.com/typestack/class-validator#readme>

9. Step 7 — Create Services (with Repositories)

What is a Repository?

WHY: A Repository is a TypeORM object that gives you methods to interact with one specific database table. Instead of writing raw SQL, you call methods like `.find()`, `.save()`, `.update()`, and `.delete()`. The Repository talks directly to Oracle and returns results as TypeScript objects.

9a. Books Service

Update the Books module first to register the entity: `src/books/books.module.ts`

```
import { Module } from '@nestjs/common';  
import { TypeOrmModule } from '@nestjs/typeorm';  
import { BooksService } from './books.service';  
import { BooksController } from './books.controller';  
import { Book } from './entities/book.entity';  
  
@Module({  
  imports: [TypeOrmModule.forFeature([Book])],  
  controllers: [BooksController],  
  providers: [BooksService],  
})  
export class BooksModule {}
```

Now write the full CRUD service: `src/books/books.service.ts`

```
import { Injectable, NotFoundException } from '@nestjs/common';  
import { InjectRepository } from '@nestjs/typeorm';  
import { Repository } from 'typeorm';  
import { Book } from './entities/book.entity';  
import { CreateBookDto } from './dto/create-book.dto';  
import { UpdateBookDto } from './dto/update-book.dto';
```

```

@Injectable()
export class BooksService {
  constructor(
    @InjectRepository(Book)
    private booksRepository: Repository<Book>,
  ) {}

  // CREATE — saves a new book to the database
  async create(createBookDto: CreateBookDto): Promise<Book> {
    const book = this.booksRepository.create(createBookDto);
    return await this.booksRepository.save(book);
  }

  // READ ALL — fetches every row from the books table
  async findAll(): Promise<Book[]> {
    return await this.booksRepository.find();
  }

  // READ ONE — fetches a single book by its primary key
  async findOne(id: number): Promise<Book> {
    const book = await this.booksRepository.findOne({ where: { id } });
    if (!book) throw new NotFoundException(`Book with id ${id} not found`);
    return book;
  }

  // UPDATE — changes specific fields on an existing book
  async update(id: number, updateBookDto: UpdateBookDto): Promise<Book> {
    await this.findOne(id);
    await this.booksRepository.update(id, updateBookDto);
    return await this.findOne(id);
  }

  // DELETE — removes a book row from the database
  async remove(id: number): Promise<{ message: string }> {
    await this.findOne(id);
    await this.booksRepository.delete(id);
    return { message: `Book ${id} deleted successfully` };
  }
}

```

9b. Users Service

Update: src/users/users.module.ts

```

import { Module } from '@nestjs/common';

```

```
import { TypeOrmModule } from '@nestjs/typeorm';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';
import { User } from './entities/user.entity';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([User])],
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}
```

Write the full CRUD service: src/users/users.service.ts

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from './entities/user.entity';
import { CreateUserDto } from './dto/create-user.dto';
import { UpdateUserDto } from './dto/update-user.dto';
```

```
@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private usersRepository: Repository<User>,
  ) {}
```

```
  async create(createUserDto: CreateUserDto): Promise<User> {
    const user = this.usersRepository.create(createUserDto);
    return await this.usersRepository.save(user);
  }
```

```
  async findAll(): Promise<User[]> {
    return await this.usersRepository.find();
  }
```

```
  async findOne(id: number): Promise<User> {
    const user = await this.usersRepository.findOne({ where: { id } });
    if (!user) throw new NotFoundException(`User with id ${id} not found`);
    return user;
  }
```

```
  async update(id: number, updateUserDto: UpdateUserDto): Promise<User> {
    await this.findOne(id);
```

```

    await this.usersRepository.update(id, updateUserDto);
    return await this.findOne(id);
  }

  async remove(id: number): Promise<{ message: string }> {
    await this.findOne(id);
    await this.usersRepository.delete(id);
    return { message: `User ${id} deleted successfully` };
  }
}

```

Read more about TypeORM Repository API: <https://typeorm.io/repository-api>

Read more about NestJS services and providers: <https://docs.nestjs.com/providers>

10. Step 8 — Create Controllers

What is a Controller?

WHY: A Controller receives incoming HTTP requests, calls the appropriate service method, and sends back a response. Each method in the controller maps to an HTTP verb (GET, POST, PATCH, DELETE) and a URL route. The controller never talks to the database directly — that is the service's job.

10a. Books Controller

Replace the contents of `src/books/books.controller.ts`:

```

import { Controller, Get, Post, Body, Patch, Param, Delete } from '@nestjs/common';
import { BooksService } from './books.service';
import { CreateBookDto } from './dto/create-book.dto';
import { UpdateBookDto } from './dto/update-book.dto';

@Controller('books')
export class BooksController {
  constructor(private readonly booksService: BooksService) {}

  // POST /books — create a new book
  @Post()
  create(@Body() createBookDto: CreateBookDto) {
    return this.booksService.create(createBookDto);
  }
}

```

```

// GET /books — get all books
@Get()
findAll() {
  return this.booksService.findAll();
}

// GET /books/:id — get one book
@Get('/:id')
findOne(@Param('id') id: string) {
  return this.booksService.findOne(+id);
}

// PATCH /books/:id — update a book
@Patch('/:id')
update(@Param('id') id: string, @Body() updateBookDto: UpdateBookDto) {
  return this.booksService.update(+id, updateBookDto);
}

// DELETE /books/:id — delete a book
@Delete('/:id')
remove(@Param('id') id: string) {
  return this.booksService.remove(+id);
}
}

```

10b. Users Controller

Replace the contents of `src/users/users.controller.ts`:

```

import { Controller, Get, Post, Body, Patch, Param, Delete } from '@nestjs/common';
import { UsersService } from './users.service';
import { CreateUserDto } from './dto/create-user.dto';
import { UpdateUserDto } from './dto/update-user.dto';

@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Post()
  create(@Body() createUserDto: CreateUserDto) {
    return this.usersService.create(createUserDto);
  }

  @Get()

```

```

    findAll() {
      return this.userService.findAll();
    }

    @Get('/:id')
    findOne(@Param('id') id: string) {
      return this.userService.findOne(+id);
    }

    @Patch('/:id')
    update(@Param('id') id: string, @Body() updateUserDto: UpdateUserDto) {
      return this.userService.update(+id, updateUserDto);
    }

    @Delete('/:id')
    remove(@Param('id') id: string) {
      return this.userService.remove(+id);
    }
  }
}

```

Read more about NestJS controllers: <https://docs.nestjs.com/controllers>

11. Step 9 — Start the Application & Test with Postman

Start your application:

```
npm run start:dev
```

Watch the terminal. You should see TypeORM log messages showing it is connecting to Oracle and creating or synchronizing the BOOKS and USERS tables. If you see any connection errors go back to Step 1 and verify your PDB is open.

11a. Books Endpoints — Test All 5

#	Method	URL	Body (JSON)	Expected Result
1	POST	/books	{ "title": "1984", "author": "George Orwell", "year": 1949 }	Returns created book with id
2	GET	/books	None	Returns array of all books

#	Method	URL	Body (JSON)	Expected Result
3	GET	/books/1	None	Returns single book with id 1
4	PATCH	/books/1	{ "genre": "Dystopian" }	Returns updated book
5	DELETE	/books/1	None	Returns success message

11b. Users Endpoints — Test All 5

#	Method	URL	Body (JSON)	Expected Result
1	POST	/users	{ "name": "Alice Banda", "email": "alice@lib.com", "role": "admin" }	Returns created user with id
2	GET	/users	None	Returns array of all users
3	GET	/users/1	None	Returns single user with id 1
4	PATCH	/users/1	{ "role": "member" }	Returns updated user
5	DELETE	/users/1	None	Returns success message

How to send a POST request in Postman:

1. Open Postman and click New then HTTP Request
2. Set the method dropdown to POST
3. Enter the URL: `http://localhost:3000/books`
4. Click the Body tab, select raw, then choose JSON from the dropdown
5. Paste the JSON body shown in the table above
6. Click Send and inspect the response

Read more about Postman requests:

<https://learning.postman.com/docs/getting-started/first-steps/sending-the-first-request>

12. Step 10 — Verify Data in Oracle Using SQL*Plus

What are we doing?

WHY: Now that you have made API calls through Postman, you should go into Oracle and run SQL queries to confirm that the data actually arrived in the database tables. This is a critical skill — always verify your application data at the database level.

Open SQL*Plus and connect as your library user:

```
sqlplus library_user/library_password@localhost:1521/library_pdb
```

12a. Verify Books

Check all books were inserted:

```
SELECT * FROM books;
```

Check a specific book:

```
SELECT * FROM books WHERE id = 1;
```

Check the update worked (genre column):

```
SELECT id, title, genre FROM books;
```

Confirm a deleted book is gone:

```
SELECT COUNT(*) FROM books;
```

12b. Verify Users

```
SELECT * FROM users;
```

```
SELECT id, name, email, role FROM users WHERE role = 'admin';
```

Check that deleting a user actually removed the row:

```
SELECT COUNT(*) FROM users;
```

12c. Check Table Structure

You can also inspect the table columns TypeORM created:

```
DESCRIBE books;
```

```
DESCRIBE users;
```

NOTE: Run these SQL queries after EVERY Postman test and include screenshots of the SQL*Plus output in your submission. This proves the data is really reaching Oracle.

Read more about SQL SELECT statements:
<https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/SELECT.html>

13. All Endpoints Summary

Module	Method	Endpoint	Action
Books	POST	/books	Create a new book in the database
Books	GET	/books	Fetch all books from the database
Books	GET	/books/:id	Fetch one book by its ID
Books	PATCH	/books/:id	Update one or more fields on a book
Books	DELETE	/books/:id	Delete a book from the database
Users	POST	/users	Create a new user in the database
Users	GET	/users	Fetch all users from the database
Users	GET	/users/:id	Fetch one user by their ID
Users	PATCH	/users/:id	Update one or more fields on a user
Users	DELETE	/users/:id	Delete a user from the database

14. Further Reading & Documentation

NestJS

NestJS Official Documentation: <https://docs.nestjs.com>

NestJS Controllers: <https://docs.nestjs.com/controllers>

NestJS Providers & Services: <https://docs.nestjs.com/providers>

NestJS Modules: <https://docs.nestjs.com/modules>

NestJS Database (TypeORM): <https://docs.nestjs.com/techniques/database>

NestJS Validation (DTOs): <https://docs.nestjs.com/techniques/validation>

NestJS Configuration (.env): <https://docs.nestjs.com/techniques/configuration>

NestJS Exception Handling: <https://docs.nestjs.com/exception-filters>

TypeORM

TypeORM Official Documentation: <https://typeorm.io>

TypeORM Entities: <https://typeorm.io/entities>

TypeORM Repository API: <https://typeorm.io/repository-api>

TypeORM Data Source Options: <https://typeorm.io/data-source-options>

Oracle Database

Oracle Pluggable Databases Guide:

<https://docs.oracle.com/en/database/oracle/oracle-database/21/multi/introduction-to-the-multitenant-architecture.html>

Oracle SQL SELECT Reference:

<https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/SELECT.html>

Oracle CREATE USER Statement:

<https://docs.oracle.com/en/database/oracle/oracle-database/21/sqlrf/CREATE-USER.html>

Supporting Libraries

class-validator decorators: <https://github.com/typestack/class-validator#readme>

Postman — Sending Requests:

<https://learning.postman.com/docs/getting-started/first-steps/sending-the-first-request>